

Seminario de Ciencias de la Computación “A”

Modelación y simulación computacional basada en Agentes 2026-I

Práctica 1: “Implementación y análisis de autómatas celulares”

Las prácticas se componen de una parte de implementación y una parte de análisis. Es necesaria una buena implementación para contestar las preguntas del análisis. En cada ejercicio se muestra la puntuación. La parte que se evalúa principalmente es la solución a los ejercicios a través de un documento-reportero donde explicarán lo que se pide en cada inciso.

Parte 1. Preguntas (1 pt)

Conteste las siguientes preguntas a partir de lo visto en la clase.

- ¿Qué es la modelación basada en agentes?
- ¿Qué es el enfoque bottom-up?
- ¿Cuándo se usa el concepto de autoorganización?
- ¿Qué es una propiedad emergente?

Parte 2. Autómata Celular Elemental (5 pts.)

Desarrollar un autómata celular elemental con el lenguaje de programación de su elección o utilizar como base el código en NetLogo desarrollado en las ayudantías. Implementar las siguientes características (1 pt):

- Función de transición.** El ACE está definido por una vecindad de $\text{radio}=1$ y un alfabeto $k = \{0,1\}$ generando 256 posibles reglas (o tablas de transición). Implementar la función de transición la cual recibe como input el número de la regla en decimal para establecer los valores de la tabla de transiciones en binario.
- Fronteras.** Implementar los dos tipos de fronteras, periódicas (toro) y fijas (con frontera definida).
- Tamaño.** Definir la longitud y el tiempo de evolución como parámetros para la construcción del ACE. Para una correcta visualización se recomienda un tamaño mínimo de 100 celdas por 100 tiempos de evolución.
- Condiciones iniciales.** Definir la posibilidad de iniciar el autómata con condiciones aleatorias, simples (fija) o introducida por el usuario.
- Salida gráfica.** Como *input* del programa se introduce el número de regla en el intervalo $[0,255]$ y como *output* la visualización gráfica.

Ejercicios:

- Exploración.** Ejecute algunas de las 256 reglas del autómata celular a partir de la implementación de su programa con condiciones de frontera periódicas (cilindro), mínimo 100 tiempos de evolución y condiciones iniciales aleatorias. Identifique el tipo de dinámica que presenta y a que clase pertenece según las cuatro categorías de la Clasificación de Wolfram. Muestre 1 representante de cada clase, adjunte la “captura de pantalla” y explique por qué consideran que pertenece a tal clase. Después de explorar todas o algunas de las 256 reglas con el programa que implementaste ¿Cuántos autómatas hay en cada clase según su dinámica? (1 pt).
- Sensibilidad a las condiciones iniciales.** Encuentre un autómata de clase III, perturbe la condición inicial aleatoria por un bit, evolucione y trate de ver las diferencias entre el autómata perturbado y el normal. **Hint:** pueden tomar las diferencias entre los estados de las celdas y pintarlos en la posición correspondiente. ¿La sensibilidad a las condiciones iniciales es una propiedad suficiente para catalogar la dinámica como caótica? Explique. Adjunte las imágenes del ACE perturbado, sin perturbar y la diferencia de estados (1pt).
- ¿Qué poder de cómputo tiene la regla 132? Explique y adjunte captura de pantalla. ¿Qué propiedades tiene la regla 30? Explique y adjunte capturas de la simulación (0.5 pt).
- En las siguientes imágenes se muestra la evolución de la regla 22 con condición inicial simple (una celda negra en el centro, Figura A) y condición inicial aleatoria (Figura B). Explique lo que sucede con la dinámica (1 pt).

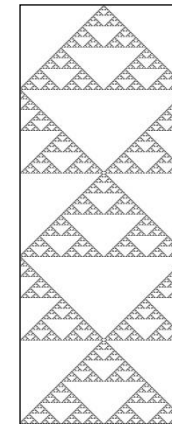


Figura A

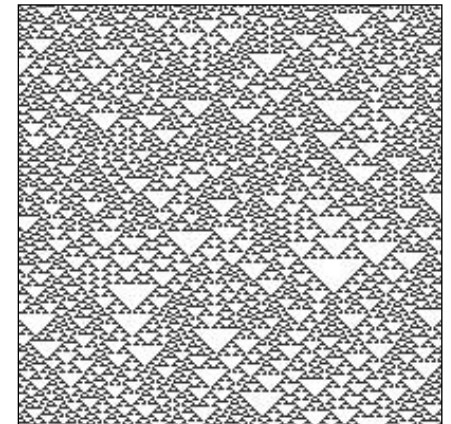


Figura B

- Si el autómata estuviera definido en un alfabeto de 3 estados considerando sus primeros vecinos, ¿cuántas posibles reglas hay? (0.5 pt).

Parte 3. LIFE: EL JUEGO DE LA VIDA (4 pts.)

El juego de la vida (LIFE), propuesto John Conway, es uno de los ejemplos más representativos de un autómata celular bidimensional. Su conjunto de reglas genera una gran cantidad de patrones, que aún hoy en día, se siguen encontrando.

Implemente LIFE utilizando el código visto en clase o con el lenguaje de su elección. Considere, las siguientes reglas de actualización con una vecindad de Moore

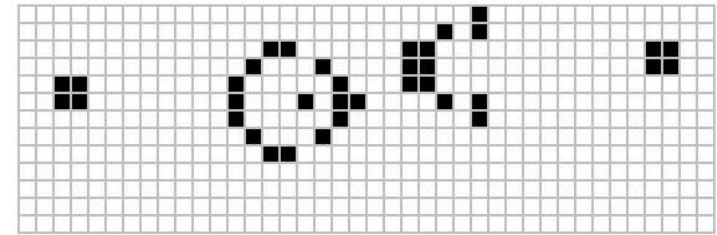
1. Para una celda que esta "viva"
 - a) Si tiene uno o cero vecinos "muere" (por soledad).
 - b) Si tiene cuatro o más vecinos "muere" (por sobrepoblación).
 - c) Si tiene dos o tres vecinos "sobrevive"
2. Para una celda que esta "vacía" o "muerta"
 - a) Si tiene tres vecinos la celda "vive".

El mínimo del tamaño de la retícula debe ser de 100x100. Implementar condición de frontera periódica y no periódica.

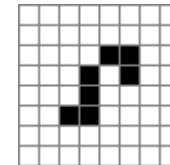
Ejercicios:

1. En NetLogo construya una gráfica de conteo de celdas vivas a través del tiempo. Ejecute varias veces su programa con una retícula de 100x100 celdas y condición aleatoria con 50% de celdas ocupadas, ¿en cuánto tiempo (ticks) se "estabilizó" la dinámica?, ¿para cualquier configuración inicial aleatoria pasará lo mismo? Explique y adjunte una captura de pantalla del "view" de agentes y la gráfica de conteo **(0.5 pt)**. Realice el mismo experimento aumentando la densidad de la condición aleatoria a 85%, ¿sucede lo mismo? Explique **(0.5 pt)**.
2. Dados los siguientes elementos implemente la compuerta NOT con el diagrama de la Figura D) **(2pt)**.

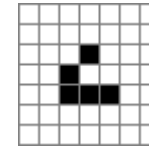
A) Gosper's gun



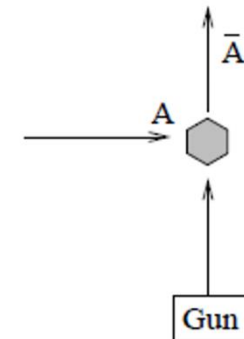
B) Eater



C) Glider



D) Diagrama

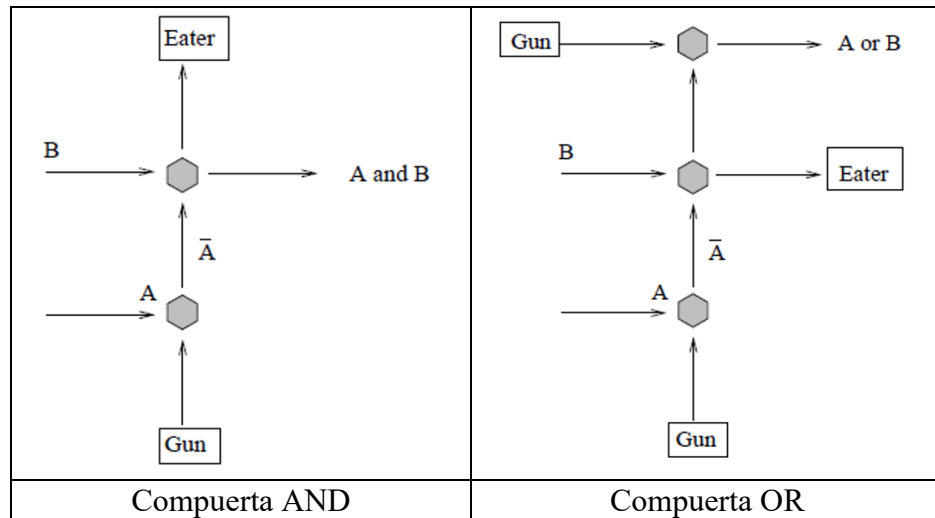


¿Qué implicaciones tiene controlar ciertas estructuras en función del tiempo?

3. ¿El juego de la vida es reversible? Explique, muestre ejemplos. **(1pt)**.

Parte 4. CIRCUITOS LÓGICOS EN LIFE (1 pt extra)

Con el uso de los elementos de la parte anterior (pistola, gliders, comedor), implemente uno de los circuitos lógicos: AND y OR. Te puedes apoyar de los diagramas siguientes. Reporte y explique la secuencia del circuito con el uso de screenshots.



Hint: Consulta el siguiente video para darte idea de la posición de los elementos en el entorno <https://www.youtube.com/watch?v=Kk2MH9O4pXY>.

Lineamientos para la entrega

La práctica se realiza de manera individual.

Entregar un empaquetado (.tgz, .zip, etc.) con la siguiente estructura:

```
ApellidoPaterno_ApellidoMaterno
|----- Practica1
|----- Fuentes
|----- readme.txt
|----- solucion.pdf
```

Fuentes: código del programa, ejecutables, etc.

Readme.txt: explicar brevemente como se ejecuta el programa, y la forma para cambiar valores de los parámetros.

Solucion.pdf: En este documento incluir todas las respuestas de la práctica junto con las gráficas y comentarios extras (si los hay). Este es el documento que se evaluará principalmente.

Subir la práctica en el Google Classroom:

<https://classroom.google.com/c/NzczODc4MzUzNTU0>

Fecha límite de entrega: 18 de septiembre de 2025.

Las prácticas que se entreguen después de la fecha de entrega tendrán una penalización en la calificación, cada día de retraso se le bajará 0.5 de la calificación obtenida.

Se penalizará utilizar código obtenido con generadores de contenido de IA o bajado de algún repositorio de la red.

Contacto

Gustavo Carreón gcarreon@unam.mx

Marco Antonio Jiménez Limas marcojimenez@ciencias.unam.mx